Approximation Algorithm

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

December 26, 2024

Overview

1. Approximation Algorithms
2. Vertex Cover Problem
3. Travelling Salesperson Problem (TSP)
4. Bin Packing Problem
5. k center problem
6. Books etc.

Introduction

- Need to find some solutions for NP complete problems, ie, solutions for the optimization problems. To cope up with NP-completeness, use
- **Brute force search:** used for small input size, $n \leq 20$
- **Heuristics:** produces valid solution, but no gurantee on optimality
- **General search Algorithms:** use Branch-and-bound, Metropolis-Hastings, simulated Annealing, Genetic algorithm, Ant colony optimization, Particle swarm optimization and many more; performances of the algorithms varies from problem to problem and instance to instance, but usually perform well
- **Approximation Algorithm:** These algorithm runs in polynomial time ideally and produces a solution that is **guranteed** to be **within some factor** of optimum solution

Performance Bounds

How good is approximation algorithm?

- **Performance ratio** of solution of approximation algo with respect to optimal solution is obtained
- Given an instance I of a problem, let $C(I)$ be the cost of the solution obtsained by the approximation algorithm and let $C^*(I)$ be the optimal solution
- **For minimization problem:** $\frac{C(I)}{C^*(I)} \geq 1$
- **For maximization problem:** $\frac{C^*(I)}{C(I)} \geq 1$
- Thus, for input size n, $\forall I, I = n$, **perormance ratio bound** $\rho(n)$ is

$$\max_I \left(\frac{C(I)}{C^*(I)}, \frac{C^*(I)}{C(I)} \right) \leq \rho(n)$$

- $\rho(n) = 1$, if and only if approximate solution is true optimum solution

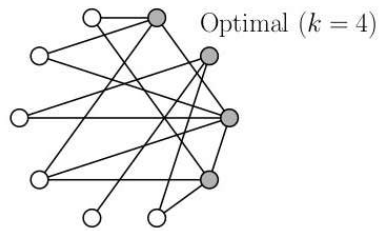
Types of Approximation ratio

- Some NP-complete problems are **inapproximable**, ie, no polynomial time algorithm achieves a ratio bound smaller than ∞
- Some NP-complete problems can be **approximated** , but **ratio bound is a function of** n
- Some NP-complete problems can be **approximated** and ratio bound is **constant**
- Some NP-complete problems can be **approximated arbitarily well**; given a parameter $\epsilon > 0$, algo. acheives a **ratio bound of** $(1+\epsilon)$; if ϵ approaches 0, running time of algo gets worse; but if it runs in polynomial time for any fixed ϵ, it is called **Polynomial Time Approximation Scheme (PTAS)**

Vertex cover problem and a heuristic

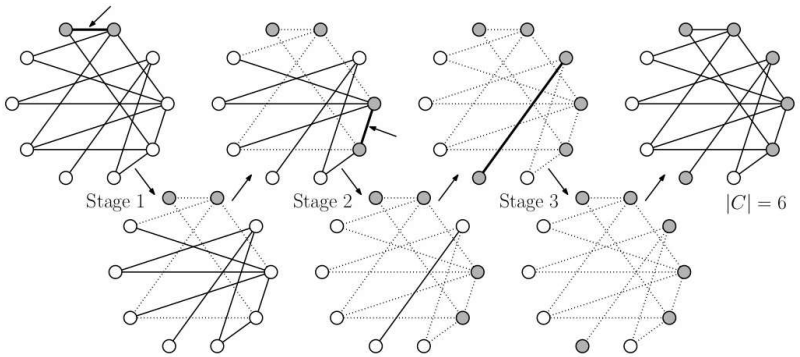
Vertex Cover is a subset of vertices such that every edge in the graph is incident to at least one of these vertices.
Vertex Cover Problem (VC): Find a vertex cover of minimum size
A *reasonably* good strategy, a *heuristic* is

- Initially the cover is empty
- During each stage, select an arbitrary edge (u, v) from the graph G and **add both** u **and** v to the current cover
- Remove all the edges that are incident to either u or v (since these edges are now all covered)
- repeat until G has no more edges.



Vertex cover: 2-for-1 heuristic

```
two-for-one-VC(G=(V,E)) {
    C = empty
    while (E is nonempty) do {
(*)     let (u,v) be any edge of E
        add both u and v to C
        remove from E all edges incident to either u or v
    }
    return C
}
```



Approximation ratio for 2-for-1 heuristic

Claim: The 2-for-1 approximation for VC achieves a performance ratio of 2
Proof:

- The vertex cover obtained for G is at most twice the size of the optimal vertex cover.
- Let set C is **output** of two-for-one-VC(G) and let C^* be the **optimum** vertex cover.
- Let A be the set of edges selected by the line marked $*$ in code
- Because we add both endpoints of each edge of A to C, we have $|C| = 2|A|$.
- However, the optimum vertex cover C^* must contain at least one of these two vertices; Therefore, $|C^*| \geq |A|$
- Therefore, $|C| = 2|A| \leq 2|C^*| \implies \frac{|C|}{|C^*|} \leq 2$

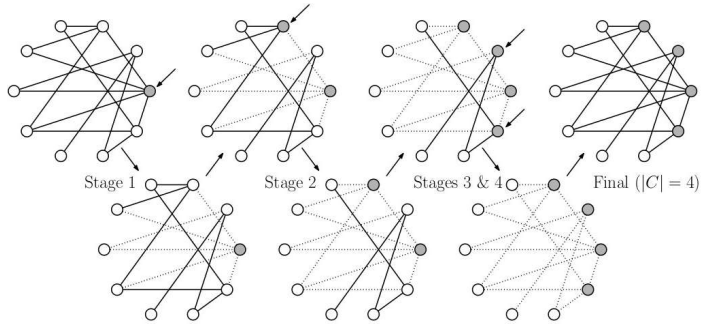
Approximation Algorithm: Proof strategy

- need some way of finding a bound on the optimal solution
- **For minimization problems:** find a **lower bound**
- **for maximization problems:** find an **an upper bound**
- The bound must be computed in polynomial time; ie, for vertex cover, the bound is related to the set of edges A, which form a *maximal independent set* of edges.

Vertex cover problem: A greedy heuristic

An improvement of 2-to-1 heuristic

```
greedy-VC(G=(V,E)) {
    C = empty
    while (E is nonempty) do {
        let u be the vertex of maximum degree in G
        add u to C
        remove from E all edges incident to u
    }
    return C
}
```



Notes on approximation algorithms

- Though greedy heuristic for VC problem gave optimal result, it **does not achieve a constant performance bound**
- However, experimental studies show that greedy actually works quite well in practice, and for *typical* graphs, it performs better than the 2-for-1 heuristic

Reductions and Approximations:

- Existence of an approximation algo. for Vertex cover problem (an NP complete problem) does not imply, all NP complete problems will have factor 2 approximation
- If V' is a VC of size k, then complement vertex set $V \setminus V'$ is an Independent Set of size $n - k$
- 2-to-1 approximation algo. is guaranteed to produce a vertex cover V'' that is of size at most $2k$, ie, by complementing our approximation $V \setminus V''$ we have an **approximate** independent set of size $n - 2k$
- Approximation Ratio bound for Independent Set is $\rho(n, k) = \frac{n-k}{n-2k}$
- The ratio could be arbitrarily large, which does not make any sense.

Travelling Salesperson Problem (TSP)

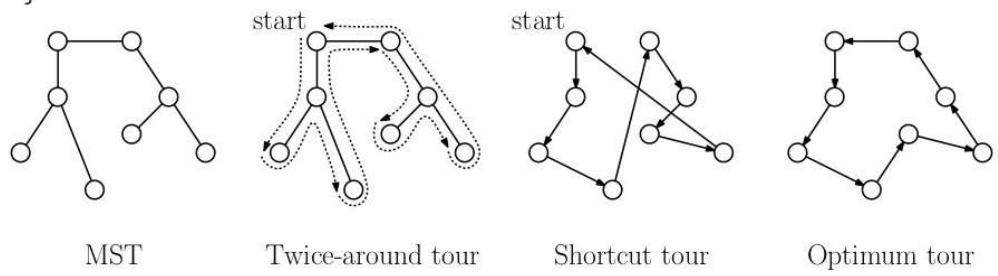
- **Traveling Salesperson Problem (TSP):** Given a complete undirected graph G with nonnegative edge weights, find a **cycle** that visits **all vertices** and is of **minimum cost**
- Let $w(u, v)$ denote the weight on edge (u, v). Given a set of edges A forming a tour, we define $W(A)$ to be the sum of edge weights in A
- **Triangle Inequality:** The direct path from u to x, is never longer than an indirect path. More formally, $\forall u, v, x \in V$, $w(u, v) \leq w(u, x) + w(x, v)$
- **Graphs satisfying Triangle Inequality:**
  - When $w(u, v)$ is defined as **shortest path length** between u and v
  - Given a set of points in the plane, and we define a **complete graph** on these points, where $w(u, v)$ is defined to be the Euclidean distance between u and v
- If the given graph for TSP satisfies Triangle Inequality, there exists an approximation algorithm for TSP with ratio bound of 2

Travelling Salesperson Problem (TSP): Intuition

- If an edge is removes from a cycle or tour , it becomes a **spanning tree**, but may not be a minimum spanning tree (MST)
- Thus, the cost of the tour is atleast as large as cost of MST
- Hence compute MST using Kruskal's or Prim's algorithm
- Convert the MST into a TSP tour while increasing its cost by at most a constant factor
- If the edge weights satisfy triangle inequality, then it is possible
- **Approx. algo:**
  - Given any free tree, there is a tour of the tree called a twice around tour that traverses the edges of the tree twice, once in each direction
  - Paths between revisited vertices are made simple by short-cutting, that is, skip over previously visited vertices
  - Notice that, final order in which vertices are visited using the short-cuts is exactly the same as a preorder traversal of the MST
  - Triangle inequality assures that the path length will not increase for taking short-cuts

TSP: Approximation algorithm

```
approx-TSP(G=(V,E)) {
    T = minimum spanning tree for G
    r = any vertex
    H = list of vertices visited by a preorder walk of T starting at r
    return L
}
```



TSP: Approximation ratio

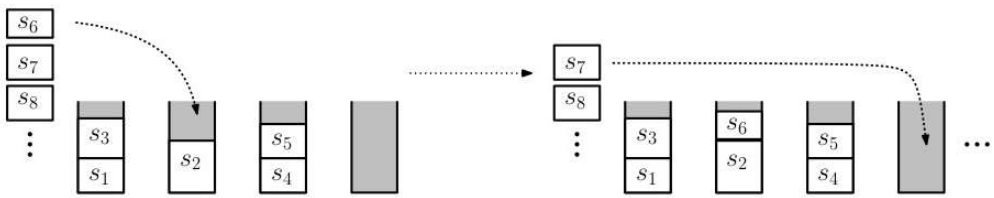
Claim: Approx-TSP achieves a performance ratio of 2
Proof:

- Let H denote the tour produced by approx. algorithm and let H^* be the optimum tour
- Let T be the **minimum spanning tree**
- Removal of any edge of H^* results in a **spanning tree**, and since T is the minimum cost spanning tree, we have $W(T) \leq W(H^*)$
- Twice around tour of T has cost $2.W(T)$, since every edge in T is hit twice
- By the triangle inequality, when we short-cut an edge of T to form H there is no increase in cost of the tour, and so $W(H) \leq 2.W(T)$
- Combining, $W(H) \leq 2.W(T) \leq 2.W(H^*) \implies \frac{W(H)}{W(H^*)} \leq 2$,

Bin Packing Problem

Bin Packing: Given a set of n objects, where s_i denotes the size of the ith object, $0 < s_i < 1$, and bins of capacity at most 1, the problem is to partition the objects among the bins such that number of bins is minimized

- bin Packing is an NP complete problem, can be reduced from Knapsack problem
- **Approx. algo**:first-fit heuristic
  - Start with an unlimited number of empty bins
  - Take each object in turn, and find the first bin that has space to hold this object and put that object in this bin.



Bin Packing: Approximation ratio (1)

Claim: First-fit uses at most twice bins as the optimum. That is, if the optimal solution uses b_{opt} bins, and first-fit uses b_{ff} bins, then $\frac{b_{ff}}{b_{opt}} \leq 2$
Proof:

- Consider an instance $\{s_1, \cdots, s_n\}$ of the bin packing problem. Let $S = \sum_i s_i$ denote the sum of all the object sizes
- Since no bin can hold more than one unit's worth of items, and we have a total of S units to be stored, it follows that we need a minimum of S bins to store everything (ie, every bin were filled exactly to the top). Thus, $b_{opt} \geq S$.
- Next, we claim that $b_{ff} \leq 2S$. How?
  - Let t_i denote the total size of the objects that first-fit puts into bin i.
  - There cannot be two bins $i < j$ such that $t_i + t_j < 1$. Because any item we decided to put into bin j must be small enough to fit into bin i
  - Thus, the first-fit algorithm would never put such item into bin j
  - This implies that $\forall i, t_i + t_{i+1} \geq 1$ (where indices are taken circularly modulo the number of bins)

Bin Packing: Approximation ratio (2)

Claim: First-fit uses at most twice bins as the optimum. That is, if the optimal solution uses b_{opt} bins, and first-fit uses b_{ff} bins, then $\frac{b_{ff}}{b_{opt}} \leq 2$

- $b_{ff} = \sum_{i=1}^{b_{ff}} 1 \leq \sum_{i=1}^{b_{ff}} (t_i + t_{i+1}) = \sum_{i=1}^{b_{ff}} t_i + \sum_{i=1}^{b_{ff}} t_{i+1} = S + S = 2S \leq 2.b_{opt}$
- Thus, $\frac{b_{ff}}{b_{opt}} \leq 2$

k Center Problem

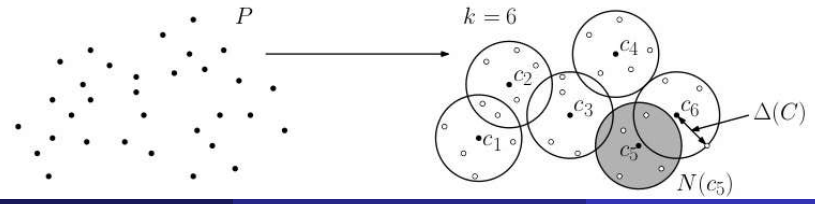
k **center problem:** Given a set P of n points in space, for each pair of points $(u, v) \in P$, let $\delta(u, v)$ denote the distance between u and v. Assume that the distance satisfies the typical properties of any natural distance function:

- **Positivity:** $\delta(u, v) \geq 0$ and $\delta(u, v) = 0$ if and only if $u = v$.
- **Symmetry:** $\delta(u, v) = \delta(v, u)$
- **Triangle inequality:** $\delta(u, w) \leq \delta(u, v) + \delta(v, w)$

The objective is to identify a subset C of k points of P , called cluster centers, such that the maximum distance of every point of P to its closest point in C is minimized. Set C is called the Cluster centers.

k Center Problem : Geometric view

- Given a set $C = \{c_1, \cdots, c_k\}$ of cluster centers and $c_i \in C$ its neighborhood, denoted as $N(c_i)$, is the set of points u of P such that c_i is the closest center to u, that is $N(c_i) = \{v \in P | \delta(v, c_i) \leq \delta(v, c_j), \text{for} i \neq j\}$
- Every point of $N(c_i)$ lies within distance $\Delta(C)$ of c_i
- if Euclidean distance is considered and P are points in the plane, then every point of $N(c_i)$ lies within a circular disk of radius $\Delta(C)$ centered at c_i
- **k-center problem:** Determine the smallest radius r such that all the points of P are contained within k disks of radius r, where each disk is centered at some point of P, that is the radius $r = \Delta(C)$ and the centers of the disks are the cluster centers



k Center Problem : Geometric view

- $N(c_1), \cdots, N(c_k)$ defines a partition of P
- The **bottleneck distance** associated with each center is defined to be the distance to its farthest point in $N(c_i)$, that is, $\delta(c_i) = \max_{v \in N(c_i)} \delta(v, c_i)$.
- Finally, the overall bottleneck distance is $\Delta(C) = \max_{c_i \in C} \Delta(c_i)$
- $\Delta(C)$ is the maximum distance of any vertex from its nearest center.
- Example: If cluster centers are the locations of a Store/ Mall, then each customer (point in P) is associated with the closest Store/Mall. The set $N(c_i)$ are the customers that go to the ith Store/ Mall location. The value $\Delta(c_i)$ is the maximum distance any of these customers needs to travel, and $\Delta(C)$ is the maximum distance that anyone needs to travel to their nearest Store/ Mall

k Center Problem : Greedy Approximation

Note When k is a fixed constant, the problem can be solved in $O(n^{k+1})$ time, which is a polynomial.

Objective: Given P and k, give an algorithm that produces a set of centers such that the final **bottleneck distance** is **at most twice** the optimum value

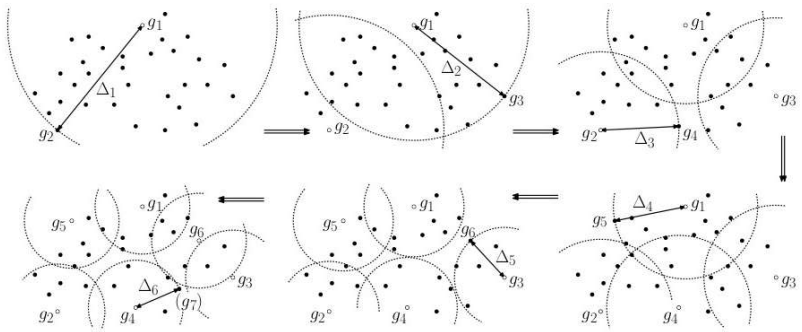
Intuition for Approximation Algorithm:

- Select any point of P to be the initial center g_1 .
- Let $G_i = \{g_1, \cdots, g_i\}$ denote the current set of centers
- Recall that $\Delta(G_i)$ is the **maximum distance** of any point of P from its nearest center, and let u be the point achieving this distance
- The greediest way to satisfy u is to put a center directly at u, that is, set $g_{i+1} \leftarrow u$, that is set $G_{i+1} \leftarrow G_i \cup \{u\}$
- The point u selected in each step is the most dissatisfied customer, as that person has to go farthest to get to the existing nearest Store. However, the algo. produces a new store coming up near his/her house.
- Repeat the process until there are k centers.

k Center Problem : Greedy Approximation

```
GreedyKCenter(P, k) {
    G = empty
    for each u in P do                  // initialize distances
        d[u] = INFINITY

    for (i = 1 to k) {
        Let u be the point of P such that d[u] is maximum
        Add u to G                      // u is the next cluster center
        for (each v in P) {             // update distance to nearest center
            d[v] = min(d[v], delta(v,u))
    }
    return G                            // final centers
}
```



Time complexity: $O(kn)$

k Center Problem : Approach to find Approximation bound

- Let $G = \{g_1, \ldots, g_k\}$ denote some set of centers computed by the greedy algorithm, and let $\Delta(G)$ denote its bottleneck distance. Let $O = \{o_1, \cdots, o_k\}$ denote any optimum set of centers, and let $\Delta(O)$ denote the optimum bottleneck distance. **To show :** $\Delta(G) \leq 2\Delta(O)$, ie., greedy approximation algorithm is within a factor of 2 of the optimum.
- Present a **lower bound** on the **optimum bottleneck distance**, and show that greedy algorithm produces a value that is at most twice this optimum value.
- For ease of analysis, consider the next center to be added after k centers as g_{k+1}. It is the point of P that maximizes the distance to its closest center in G_k, that is $\Delta(G_k)$. Also, define $G_{k+1} = \{g_1, \cdots, g_{k+1}\}$.

k Center Problem : Lemmas for Approximation bound

- **Lemma 1:** The sequence of bottleneck distances $< \Delta_1, \cdots, \Delta_k >$ in the greedy algorithm is monotonically nonincreasing, that is $\Delta_i \geq \Delta_{i+1}$ (radii of disks decreasing)
- **Lemma 2:** Every pair of greedy centers is separated by at least the greedy bottleneck dis- tance, that is, for $1 \leq i < j \leq k+1$, $\delta(g_i, g_j) \geq \Delta(G)$. (no disk in the final picture contains the center of any other.)
- **Lemma 3:** Let $\Delta_{min} = \Delta(G)/2$. Then for any set C of k cluster centers, we have $\Delta(C) \geq \Delta_{min}$ (It is not possible to cover all the points of P using k disks whose radii are smaller than Δ_{min})

k Center Problem : Proofs of Lemma (1)

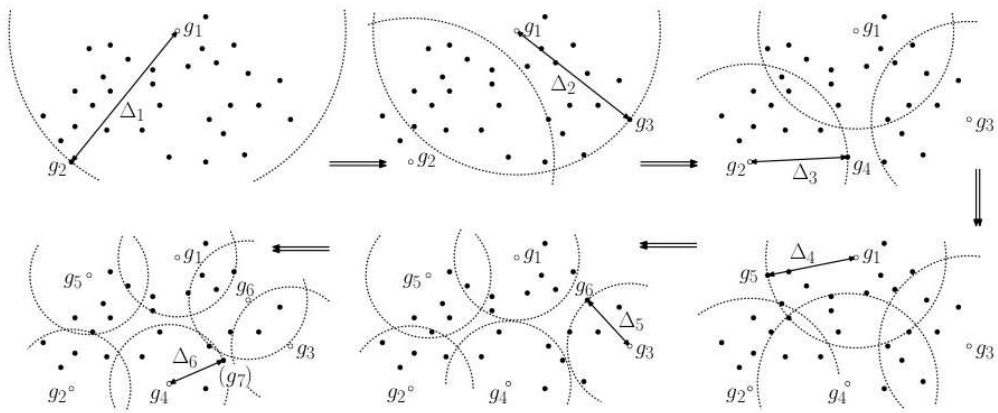
- By Lemma 3, **the bottleneck cost** of any k-center solution C must be at least as large as $\Delta_{min} = \Delta(G)/2$; Thus, Δ_{min} plays the role of the lower bound
- Since this applies to the optimal solution as well, we have $\Delta(O) \geq \Delta(G)/2$, or equivalently $\Delta(G) \leq 2\Delta(O)$. Thus, G is a factor-2 approximation to the optimal bottleneck distance.
- **Proof of Lemma 1:** As more centers are added, the distance of any point to its closest center cannot increase (it can only stay the same or decrease). Since $G_i \subseteq G_{i+1}$, we have $\Delta_i \geq \Delta_{i+1}$.
- **Proof of Lemma 2:** When g_j is added, it is at distance Δ_{i-1} from its closest center in G_{j-1}. Since $i < j$, $g_i \in G_{j-1}$, and therefore $\delta(g_i, g_j) \geq \Delta_{i-1}$, and by Lemma 1, this is at least as large as $\Delta_k = \Delta(G)$.

k Center Problem : Proofs of Lemma (2)

- **Proof of Lemma 3:** Consider any set C of k centers whose bottleneck distance is smaller than Δ_{min} .
- By the pigeonhole principal, there exists at least two centers $g_i, g_j \in G_{k+1}$, where $1 \leq i < j \leq k+1$ that are in the same neighborhood of some center $c \in C$.
- Since $\Delta(c) \leq \Delta_{min}$, by combining Lemma 2 with the triangle inequality we have $\Delta(G) \leq \delta(g_i, g_j) \leq \delta(g_i, c) + \delta(c, g_j) < \Delta(C) + \Delta(C) = 2\Delta(C)$. Therefore, $\Delta(C) > \Delta(G)/2 = \Delta_{min}$.
- Since this holds for any set of centers, it holds for the optimum set O.

the pigeonhole principle states that if n items are put into m containers, with $n > m$, then at least one container must contain more than one item.

k Center Problem



References

avid Mount, Lecture notes on Design and Analysis of Computer Algorithms, 2015, Lecture 11, 25, 37

The End